

7.10 Subroutine Uniform Variables

Subroutine uniform variables are similar to uniform variables, except they are context state rather than program state, and apply only to a single program stage. Having subroutine uniforms be context state allows them to have different values if the program is used in multiple contexts simultaneously. There is a set of subroutine uniforms for each shader stage.

A subroutine uniform may have an explicit location specified in the shader. At link time, all active subroutine uniforms without an explicit location will be assigned a unique location. The value of `ACTIVE_SUBROUTINE_UNIFORM_LOCATIONS` for a program object is the largest specified or assigned location plus one. An assigned location will never take the location of an explicitly specified location, even if that subroutine uniform is inactive. Between the location zero and the value of `ACTIVE_SUBROUTINE_UNIFORM_LOCATIONS` minus one there may be unused locations, either because they were not assigned a subroutine uniform or because the subroutine uniform was determined to be inactive by the linker. These locations will be ignored when assigning the subroutine index as described below.

There is an implementation-dependent limit on the number of active subroutine uniform locations in each shader stage; a program will fail to link if the number of subroutine uniform locations required is greater than the value of `MAX_SUBROUTINE_UNIFORM_LOCATIONS` or if an explicit subroutine uniform location is outside this limit. For active subroutine uniforms declared as arrays, the declared array elements are assigned consecutive locations.

Each function in a shader associated with a subroutine type is considered an active subroutine, unless the compiler conclusively determines that the function could never be assigned to an active subroutine uniform. The subroutine functions can be assigned an explicit index in the shader between zero and the value of `MAX_SUBROUTINES` minus one. At link time, all active subroutines without an explicit index will be assigned an index between zero and the value of `ACTIVE_SUBROUTINES` minus one. An assigned index will never take the same index of an explicitly specified index in the shader, even if that subroutine is inactive. Between index zero and the value of `ACTIVE_SUBROUTINES` minus one there may be unused indices either because they weren't assigned an index by the linker or because the subroutine was determined to be inactive by the linker. If there are no explicitly defined subroutine indices in the shader the implementation must assign indices between zero and the value of `ACTIVE_SUBROUTINES` minus one with no index unused. It is recommended, but not required, that the application assigns a range of tightly packed indices starting from zero to avoid indices between zero and the value of `ACTIVE_SUBROUTINES` minus one being unused.

To determine the set of active subroutines and subroutines used by a partic-